// DB DEEP DIVE · EP 02 · GUIDE

Database Indexing (B+ tree)

@thestackexplained
FREE PDF GUIDE

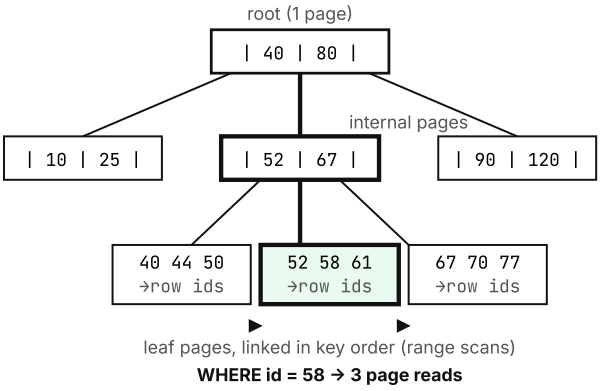
TL;DR · An index is a sorted, shallow B+ tree: O(log n) page reads instead of a full scan, paid for with storage and slower writes.

01 Core concept

Tables are stored in fixed-size **pages** (e.g. 8 KB in PostgreSQL, 16 KB in InnoDB). Without an index, `WHERE email = ?` means a **full table scan**: read every page, O(n).

A **B+ tree index** keeps the indexed values **sorted**. Each page holds hundreds of keys, so the tree is very wide and only about **4–5 levels deep** even for 50 crore rows (with ~300 keys per page: 1 root → ~19 → ~5,600 → ~17 lakh leaf pages). Internal pages route; **leaf pages** hold the keys plus a pointer to the row (or the row itself in a clustered index), and leaves are linked for range scans.

Think of a big library: floor → section → almirah → shelf → book. At every level you skip the rest of the library. That is exactly what each B-tree level does.



Lookup walks root → internal → leaf: one page per level.

02 How it works, step by step

1. Query arrives: `SELECT * FROM orders WHERE order_id = 58`; the planner sees an index on `order_id` and estimates it is selective.
2. Read the **root page**, binary-search its keys, follow the child pointer (58 lies between 40 and 80).
3. Repeat on the **internal page** until a **leaf**: about $\log_{fanout}(n)$ reads, e.g. ~300 keys per page → 4 levels cover 50 crore keys (upper levels usually cached in RAM).
4. In the leaf, find the key and its row pointer (heap TID / primary key), then fetch the row (skipped if the index **covers** the query).
5. Range query (`BETWEEN`, `>`, `ORDER BY`): find the first key, then walk linked leaves in order.
6. On INSERT/UPDATE/DELETE the DB also updates every index on that table; full pages **split**, which is why writes get slower with more indexes.

03 When does the index help?

Query / situation	Uses B+ index?	Why
`WHERE a = ?` on index (a)	Yes	Exact lookup, O(log n)
`WHERE a = ? AND b > ?` on (a, b)	Yes	Leftmost prefix, then range on b
`WHERE b = ?` on (a, b)	Usually no	Not a leftmost prefix (some DBs can skip-scan)
`WHERE LOWER(email) = ?` on (email)	No	Function on column; create an expression index
`WHERE name LIKE '%kumar'`	No	Leading wildcard; consider full-text/trigram index
Low-selectivity column (e.g. status with 3 values)	Often no	Planner prefers a scan when many rows match

04 Implementation tips

- Index what you **filter, join and sort** on; check real queries with `EXPLAIN` / `EXPLAIN ANALYZE`, not guesses.
- Composite order matters: **equality columns first, then range/sort column**, e.g. `(user_id, created_at)` for "latest orders of a user".
- Use a **covering index** (INCLUDE columns, or put them in the key) for hot read paths to avoid row lookups.
- Build indexes on big live tables without blocking writes (e.g. PostgreSQL `CREATE INDEX CONCURRENTLY`, MySQL online DDL).
- Drop unused indexes. Each one costs disk, RAM (buffer pool) and write time.

```
-- "Track order": latest orders of one user
CREATE INDEX idx_orders_user_time ON orders (user_id, created_at DESC);

EXPLAIN ANALYZE
SELECT order_id, status FROM orders
WHERE user_id = 42 ORDER BY created_at DESC LIMIT 10;
-- expect: Index Scan using idx_orders_user_time (no Seq Scan, no Sort)
```

05 Common pitfalls

- Indexing every column: writes and storage blow up, and the planner still uses only a few.
- Wrong composite order, e.g. `(created_at, user_id)` for a per-user query.
- Implicit casts or functions on the indexed column (`WHERE id = '42'` on some DBs, `DATE(created_at) = ...`) disable the index.
- Random UUIDv4 primary keys in a clustered index cause page splits and bloat; prefer time-ordered keys (UUIDv7/ULID or bigint).
- Trusting stale statistics: run ANALYZE after big loads so the planner picks the index.

06 Interview Q&A

Q. Why B+ trees and not binary search trees or hash tables?
B+ trees are wide and shallow, so few disk/page reads, and leaves are sorted and linked, which suits range queries and ORDER BY. A hash index only does equality; a BST is too deep for disk.

Q. Clustered vs non-clustered (secondary) index?
Clustered: the table rows are stored in the index's leaf order (InnoDB primary key), so there is one per table. Secondary: leaves store a pointer (row id or primary key), which needs an extra lookup.

Q. What is a covering index?
An index containing every column the query needs, so the DB answers from the index alone (index-only scan) without touching the table.

Q. Composite index (a, b, c): which WHERE clauses can use it?
Leftmost prefixes: a; a and b; a, b and c; a = ? with a range on b. Not b or c alone (except via skip scan in some engines).

Q. Why can adding an index make the system slower?
Every write must update it (with page splits), it uses buffer-pool memory, and for low-selectivity filters the planner may still scan. Read-heavy wins, write-heavy pays.

`SCENARIO` **Q. Sale day: a 50-crore-row orders table, and "Track Order" (by user, latest first) takes 4 s. What do you do?**
Run EXPLAIN: it's likely a Seq Scan + Sort. Add `(user_id, created_at DESC)`, optionally INCLUDE status, and build it CONCURRENTLY. Verify an Index Scan with no Sort and the latency in ms. Then watch write latency and replica lag. Rejected answer: "add more RAM / a bigger server".

Q. Index on a boolean or gender column: good idea?
Usually not alone (low selectivity). Use a partial index (e.g. `WHERE status = 'PENDING'`) if you query a rare value.

// QUICK REVISION

- No index → full scan, O(n) pages
- B+ tree: sorted, wide, ~4–5 levels for 50 crore rows
- Lookup = one page per level, O(log n)
- Linked leaves → fast ranges + ORDER BY
- Composite: leftmost prefix, equality first
- Covering index → no table lookup
- Functions/casts on column → index skipped
- More indexes = slower writes; verify with EXPLAIN